

1. Path Length 05A1

将路径的长度定义为**边**的总数而非**顶点**的总数，相对而言有何便利之处？

2. Depth & Height 05A1

- a) 试证明：树中任何节点 v 都满足： $depth(v) + height(v) \leq height(T)$;
- b) 何时取等号？

3. Tree Representation 05A2

在**父节点、孩子节点、父节点-孩子节点**表示法中

- a) 如何判定一对节点是否为**祖先-后代**关系？为此需要花费多少时间？
- b) 如何支持节点的**接入**？
- c) 如何支持子树的**删除**？
- d) 如何找出任意一对节点的**最低公共祖先** (Lowest Common Ancestor) ？

4. Binary Tree/Forest 05B1

我们在课上已看到，在**有根、有序**的前提下，任何一棵多叉树经过变换都对应于一棵二叉树。

- a) 反过来，任何一棵二叉树是否也会对应于某棵多叉树？
- b) 试在由有根、有序树构成的所有**有序森林**，与所有的**二叉树**之间建立一个一一对应的关系。

5. Proper Binary Tree 05B1

讲义中指出，任何一棵二叉树均可转化为一棵**真**二叉树。反过来，这种转换是否也总是可行？

6. updateHeightAbove() 05B2

为确保所有祖先的高度都能更新，示例代码中实现的算法会一直逐层上溯到根。

- a) 在某节点作为叶子插入之后，如果某个祖先的高度没有变化，是否**更高的**祖先们也必然不会变化？
- b) 在某个叶节点被删除之后呢？
- c) 在某棵子树接入 (attach()) 之后呢？
- d) 在某棵子树分离 (secede()) 之后呢？
- e) 基于以上结论，可如何**优化**updateHeightAbove()算法？
- f) 如何评估你的优化效果？

7. Energetic vs. Lazy 05B2

二叉树的height、size等信息有两种记录方式，在我们的示例代码中，前者采用“**勤奋策略**”，每个节点的高度一旦有变化，便立即更新；后者则采用“**懒惰策略**”，直到需要时才通过接口size()递归地统计。

- a) 试颠倒过来，分别修改代码，用另一种策略来记录height、size，并完成测试。
- b) 两种策略各有什么优点、缺点，分别**适用于**哪些应用场合？

8. Head/tail recursion 05C1

考查二叉树的**递归式**中序遍历算法。

- a) 对于什么样的二叉树，遍历的过程会相当于**尾递归**？
- b) 对于什么样的二叉树，遍历的过程会相当于**头递归**（递归发生在函数中**最靠前**的位置，仅次于递归基）？

9. Tail recursion**05C1**

考查二叉树的**递归式**先序遍历算法。

- 试验证：在二叉树退化成**单链**（所有节点至多只有一个孩子）时，遍历的过程相当于**尾递归**；
- 此时，时间、空间复杂度可以如何**优化**？
- 如果BinNode**没有**设置parent引用，你的优化是否依然可行？

10. BinNode::size()**05C1**

- 试验证：本节介绍的二叉树规模统计算法，除了**先序**遍历，也可以改用**中序**、**后序**或**层次**遍历；
- 试通过定义一个函数对象visit()，直接调用BinNode::traverse(x, visit())以实现同样的功能；
- 本节所举的其它应用实例，是否也可以这样**改写**？

11. clone()**05C1**

本节介绍的二叉树深复制算法，采用了**先序**遍历的策略。是否可以改用**中序**或**后序**策略？

如果可以，具体如何实现？否则，为什么不可以？

12. printBinTree()**05C1**

本节讲义仅介绍了printBinTree()算法的基本原理及梗概。试从实例代码包中找到该算法的详细代码，阅读并了解其具体而完整的实现方式，尤其是如何展示出各节点的**深度**，并绘制出**方向**正确的联边。

13. clean()**05C1**

本节介绍的二叉树清空算法，采用了**后序**遍历的策略。是否可以改用**先序**或**中序**策略？

如果可以，具体如何实现？否则，为什么不可以？

14. RPN ~ Postorder**05C1**

在本节中我们看到，只含**一元**、**二元**运算符的合法表达式总是可以转换为**二叉树**，并进而通过**后序**遍历得到对应的RPN。那么反过来，可否由PRN得出对应的二叉树呢？若可以，方法如何？若不可以，原因何在？

15. Iterative Preorder Traversal**05C2**

讲义中介绍了先序遍历的一种迭代算法，其思路是沿着**左侧通路**逐层地分解二叉树。

- 试对称地沿着**右侧通路**来做分解，实现先序遍历的另一个迭代算法；
- 相对于讲义中的算法，新的这个算法有何优点？
- 有何缺点？

16. Correctness Of Iterative Preorder Traversal**05C2**

- 试通过对二叉树的**规模**做数学归纳，证明本节给出的迭代式**先序**遍历算法是正确的。

提示：考查藤上所有节点均被访问之后的时刻——此时，原树接下来的遍历过程，可分解为对**多棵**右子树的遍历；这些子树的规模均有所**削减**，且彼此**独立**。

- 对藤的长度做数学归纳，是否同样可以证明？如果不行，试说明理由；否则，给出你的证明。

17. Empty right subtrees**05C2**

在迭代式先序遍历preorder1()算法中，内层for循环的作用是沿藤下行，并将各右子树的根节点推出栈中。

然而我们不难发现，其中有些右子树可能是**空的**。这类空的子树随后也会在主算法中出栈，虽然不致**错误**，

却毕竟是**多余的**“过门”。

- 如何**避免**此类没有实效的入栈、出栈？
- 在你改进之后，算法的**总体**效率是否会有（当然，只是常系数意义上的）提高？

18. Storage cost of iterative preorder traversal

05C2

考查本节所介绍迭代式**先序**遍历算法的**空间**复杂度。

- 试证明：该复杂度不会超过 **T 的高度**；
- 试证明：该复杂度更精确的估计是 $\mathcal{O}(\max\{la(v) \mid v \in T\})$ ，其中 $la(v)$ 为节点 v 所有**左祖先**的总数；
- 当然，后一上界不致超过前者，但二者的**差距**最大可能多大？试构造出如此极端的 T ；
- 试构造另一极端的 T 来说明：两个上界也可能**相等**；
- 两个上界之比值的**数学期望**是多大？为什么？

19. Correctness of iterative inorder traversal

05C3

我们在课上通过对二叉树的**规模**做数学归纳，证明本节给出的迭代式**中序**遍历算法是正确的。

如果改为对树的**长度**做数学归纳，是否同样可以证明？如果可行，试给出你的证明；否则，试说明理由。

20. Last inorder node

05C4

无论何种遍历，每一棵二叉树都有且仅有一个**最终**被访问的节点。当然，对于这个节点而言的`succ()`应该返回`nullptr`。阅读本节针对中序遍历所实现的`succ()`算法，理解这一功能是如何**落实**的。

21. Parent in postOrder()

05C5

讲义中实现的迭代式后序遍历算法需要借助parent信息，以**区分**“返回父节点”或“展开右子树并遍历之”这两种情况。如果BinNode结构**没有**记录parent信息，该算法可否在经过适当调整之后依然可行？如果依然可行，试说明应如何调整；否则，试解释为何不能。

22. Correctness of iterative postorder traversal

0C5

仿照课上针对**中序**遍历的方法，证明本节迭代式**后序**遍历算法是**正确**的。

23. Amortization

05C[2|3|5]

任选一种**均摊分析**的方法以证明，迭代式先序、中序、后序遍历算法的时间复杂度均为 $\mathcal{O}(n)$ 。

24. Level-order traversal

05C6

试证明**层次**遍历的如下性质，并由此确立算法的正确性及复杂度：

- 每次迭代中入队的节点（若存在），都是出队节点的**孩子**；
- 辅助队列中的各节点，在任何时刻都按**深度**单调排列，而且深度相差不超过**1层**；
- 所有节点迟早都会入队，而且更高/低的节点，更早/晚入队；更左/右的节点，更早/晚入队；
- 每次迭代中尽管入队节点数目（从0至2）不定，但总是**恰有**一个节点出队并接受访问；
- 每个节点入、出队**恰好**各一次，故知整体只需 $\mathcal{O}(n)$ 时间。

25. Storage cost of level-order traversal

05C6

- 层次遍历的**空间**成本主要消耗于辅助队列，那么这一成本与二叉树的**结构**有何关系？
- 在规模同为 n 的所有二叉树中，哪一棵的**层次**遍历需要使用**最多**的空间？

26. Complete Binary Tree**05C6**

- a) 本节指出，完全二叉树可以向量的形式，紧凑而高效地**物理**实现。试动手完成这一任务。
- b) 这种实现方式能否充分发挥**系统缓存**的作用？也就是说，连续访问的节点是否在物理上通常会彼此**临近**？

27. Rebuilding a proper binary tree**05C7**

本节指出，仅凭其**先序**与**后序**遍历序列，依然可以重建一棵**真**二叉树，试编程实现这一算法。

28. Rebuild by augmented sequences**05C7**

- a) 试证明：由**先序**增强序列，可以构造出对应的二叉树；
- b) 试编程**实现**对应的算法；
- c) **中序**增强序列呢？
- d) **后序**增强序列呢？

29. More rebuildings**05C7**

各种遍历序列的**其它**组合，是否可以完成对原二叉树的重建？试逐一考查验证。

30. Greedy Huffman: lower/higher frequency lower/higher**05D1**

- a) 试证明Huffman贪心策略的正确性：
任何一对子树**交换**位置，只要能**使频率高/低者在树中的位置更高/低**，编码成本便会**下降**。
- b) 下降的**数值**取决于哪些因素？具体等于多少？

31. Huffman tree is optimal**05D2**

试按照讲义中的思路证明：尽管最优编码树未必**唯一**，但Huffman算法所构造出来的必属**其一**。

32. Implementing Huffman algorithm**05D3**

试阅读示例代码中Huffman算法的部分，重点厘清以下方面：

- a) 算法的**主体框架**是在何处、如何描述的？
- b) 目前是如何实现Huffman**森林**的？
- c) 算法框架的描述，如何能够**独立**于具体所选用的数据结构？
- d) 日后实现Huffman森林的其他更**高级**的数据结构，是如何与此框架自然**接驳**的？

33. Huffman tree using stack and queue**05D4**

试按照本节介绍的思路，借助一个**栈**和一个**队列**，简明地实现复杂度为 $\mathcal{O}(n \log n)$ 的Huffman算法。

34. Lower bound of average complexity**05E1**

讲义中我们已借助ADT模型证明：基于比较的任何排序算法，在最坏情况下都需要 $\Omega(n \log n)$ 时间。

试证明：其平均时间复杂度也不会低于 $\Omega(n \log n)$ 。

35. Lower bound of Huffman encoding**05E2**

- a) 试通过规约证明：最优编码树的任何构造算法，时间复杂度的下界都是 $\Omega(n \log n)$ ；
- b) 试证明：Huffman编码树的任何构造算法，时间复杂度的下界都是 $\Omega(n \log n)$ 。